Cours
Circuits Logiques

PLAN DU COURS

LOGIQUE COMBINATOIRE

CHAPITRE 1 : Système de Numération et Arithmétique binaire.

CHAPITRE 2 : Eléments de L'Algèbre de Boole et opérateurs logiques élémentaires.

CHAPITRE 3 : Fonctions Booléennes.

CHAPITRE 4 : Circuit logique binaire ; Circuit arithmétique.

LOGIQUE SEQUENTIELLE

CHAPITRE 5 : Etude des bascules.

CHAPITRE 6 : Etude des registres.

CHAPITRE 7 : Etude des compteurs.

CHAP1 : Système de Numération et Arithmétique binaire

I. Systèmes de Numération :

Pour qu'une information numérique soit traitée par un circuit, elle doit être mise sous forme adaptée à celui-ci. Pour cela, il faut choisir un système de numération de base b (b : un nombre entier naturel ≥ 2).
De nombreux systèmes de numération sont utilisés en technologie numérique. Les plus courants sont les systèmes : décimal (base 10), binaire (base 2), octal (base 8) et hexadécimal (base 16).

Tout nombre N peut se décomposer en fonction des puissances entières de la base de son système de numération. L'écriture se fait du poids faible au poids fort :

Exemple d'écriture :

$12_{(10)} = 1 . 10^{1} + 2 . 10^{0}$

D'une manière générale, on aura donc :

$$N_{(b)} = \sum_{i=-m}^{n} a_i.b^i = a_n.b^n + a_{n-1}.b^{n-1} + ... + a_1.b^1 + a_0.b^0 + a_{-1}.b^{-1} + ... + a_{-m}.b^{-m}$$

Les termes de cette expression, écrite sous forme polynomiale, sont définis ainsi :

b : base du système de numération, il représente le nombre de chiffres différents qu'utilise ce système de numération.
a_i : un chiffre parmi les chiffres de la base du système de numération tel que : $0 \leq a < b$.
b^i : poids du chiffre a_i.
i : rang du chiffre a_i.

- L'information analogique (ou continue), représentée par des grandeurs physiques à variation continue, est employée dans des calculateurs analogiques.
- L'information digitale fondamentalement discontinue est une information numérisée et représentée à l'aide de grandeurs physiques ne pouvant prendre qu'un nombre fini de valeurs discrètes.

Son support élémentaire est un système à n états d'équilibre, chaque état correspond à une valeur d'information que nous appelons ''digit'' (chiffre exprimé en Anglais). Dans la pratique, ce sont des systèmes à deux états d'équilibre qui sont utilisés (d'où le nom d'information binaire) dans les calculateurs digitaux ou ordinateurs.

1) Système décimal (base 10) :

Le système décimal comprend 10 chiffres qui sont : 0,1,2,3,4,5,6,7,8,9. Ce système est appelé aussi système à base 10, il s'est imposé tout naturellement à l'homme qui possède dix doigts. Prenons l'exemple du nombre décimal 1230, que nous écrivons $N_1=1230_{10}$. L'indice 10 représente la base du système de numération dans lequel le nombre N_1 envisagé est écrit (dans le cas de la base 2 ou base 10, cet indice est parfois négligé). Ce nombre N_1 peut s'écrire sous forme polynomiale suivante :

$$N_1 = 1230_{10} = 1.10^3 + 2.10^2 + 3.10^1 + 0.10^0$$

Dans l'écriture du nombre, la position du chiffre détermine son poids. Dans cet exemple, 0 est le chiffre de poids le plus faible et 1 celui du plus fort.
De même, on peut écrire un nombre décimal à virgule N_2 sous la forme polynomiale suivante :

$$N_2 = 429{,}657 = 4.10^2 + 2.10^1 + 9.10^0 + 6.10^{-1} + 5.10^{-2} + 7.10^{-3}$$

2) Système binaire (base 2) :

Le système décimal est difficile à l'adapter aux circuits numériques. Par exemple, il est difficile de concevoir des équipements électroniques qui puissent fonctionner avec dix niveaux de tension différents pour représenter les dix chiffres décimaux. Par contre, il est très facile d'imaginer des systèmes électroniques qui fonctionnent seulement avec deux niveaux de tension. C'est la raison pour laquelle la plupart des systèmes numériques ont recours au système binaire (base 2) comme système de numération. Dans le système binaire, il n'y a que deux chiffres possibles {0,1} qui sont souvent appelés bit ''binary digit''.
Un nombre binaire $N(a_n\ a_{n-1}\ldots a_0,\ a_{-1}\ a_{-2}\ldots a_{-m})$ s'écrit en représentation polynomiale sous la forme suivante :

$$N = a_n.2^n + a_{n-1}.2^{n-1} + \ldots + a_0.2^0 + a_{-1}.2^{-1} + \ldots + a_{-m}.2^{-m}$$

L'élément a_n est le bit de poids le plus fort, noté **M.S.B** '**Most Significant Bit**', et a_{-m} est celui de poids le plus faible, noté **L.S.B** '**Least Significant Bit**'.
Ecrivons à titre d'exemple, le nombre binaire 11000_2 sous forme polynomiale :

$$N = 11000_2 = 1.2^4 + 1.2^3 + 0.2^2 + 0.2^1 + 0.2^0$$

3) Système octal (base 8):

Ce système octal, dit aussi à base 8, comprend 8 chiffres qui sont : 0,1,2,3,4,5,6,7. Les chiffres 8 et 9 n'existent pas dans cette base.
Ecrivons à titre d'exemple, les nombres $N_1 = 324_8$ et $N_2 = 6057{,}15_8$ sous la forme d'un polynôme :

$$N_1 = 324_8 = 3.8^2 + 2.8^1 + 4.8^0 = 212_{10}$$

$$N_2 = 6057{,}14_8 = 6.8^3 + 0.8^2 + 5.8^1 + 7.8^0 + 1.8^{-1} + 4.8^{-2} = 3119{,}1875_{10}$$

4) Système hexadécimal (base 16) :

Le système hexadécimal, ou base 16, contient seize éléments qui sont les dix chiffres décimaux 0,1,2,3,4,5,6,7,8,9 plus les six lettres A,B,C,D,E,F qui représentent, respectivement, 10,11,12,13,14,15.
Ecrivons, à titre d'exemples, sous forme polynomiale les nombres hexadécimaux N_1, N_2 et N_3 :

$$N_1 = 356_{16} = 3.16^2 + 5.16^1 + 6.16^0 = 854_{10}$$

$$N_2 = 2AF_{16} = 2.16^2 + 10.16^1 + 15.16^0 = 687_{10}$$

$$N_3 = 81,B_{16} = 8.16^1 + 1.16^0 + 11.16^{-1} = 129,6875_{10}$$

Le tableau 1.1 donne les équivalences entre les systèmes : décimal, binaire, octal et hexadécimal.

Décimal	Binaire	Octal	Hexadécimal
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Tableau 1.1 : Equivalence entre les systèmes : décimal, binaire, octal et hexadécimal.

II. Technique de conversion ou Transcodage :

Il s'agit du processus de conversion d'un nombre écrit dans une base b_1 à une autre base b_2.

1) Conversion d'un nombre N de base b en nombre décimal :

La valeur décimale d'un nombre N, écrit dans une base b, s'obtient par sa forme polynomiale décrite précédemment.

Exemples :

$$N_1 = 101101_2 = 1.2^5 + 0.2^4 + 1.2^3 + 1.2^2 + 0.2^1 + 1.2^0 = 45_{10}$$

$$N_2 = 6734_8 = 6.8^3 + 7.8^2 + 3.8^1 + 4.8^0 = 3548_{10}$$

$$N_3 = A73_{16} = 10.16^2 + 7.16^1 + 3.16^0 = 2675_{16}$$

2) Conversion d'un nombre décimal :

Pour convertir un nombre décimal en un nombre de base b quelconque, on divise le nombre décimal à convertir par la base b, et on conserve le reste de la division. Le quotient obtenu est divisé de nouveau par b en conservant le deuxième reste de cette division. On répète cette opération sur chaque quotient obtenu. Le nombre recherché N dans la base b, est obtenu en écrivant successivement le premier reste à la position du chiffre de poids le plus faible, le deuxième reste à la position du poids suivant, jusqu'au dernier reste à la position du poids le plus fort.

Justification de la méthode :
Soit le nombre à convertir dans le système de numération de base b.

$$N = q_0.b + r_0$$
$$q_0 = q_1.b + r_1$$
$$q_1 = q_2.b + r_2$$
.
.
$$q_{n-2} = q_{n-1}.b + r_{n-1}$$
$$q_{n-1} = 0.b + r_n$$

avec q_i : quotient de la division

r_i : reste de la division

En remplaçant la dernière expression dans celle qui la précède et ainsi de suite, on aura : $q_{n-2} = r_n.b + r_{n-1}$

$$q_{n-3} = q_{n-2}.b + r_{n-2} = (r_n.b + r_{n-1}).b + r_{n-2} = r_n.b^2 + r_{n-1}.b + r_{n-2}$$
.
.
$$q_0 = r_n.b^{n-1} + r_{n-1}.b^{n-2} + ... + r_1$$

En remplaçant l'expression de q_0 dans N, on aura alors :

$$N = r_n.b^n + r_{n-1}.b^{n-1} + ... + r_1.b + r_0.b^0 = \sum_{i=0}^{n} r_i.b^i$$

Les restes r_i sont bien les coefficients de la décomposition polynomiale, qui constituent les digits du nombre N dans la base b.

Passage vers la base 2 :

1ère Méthode : Succession de division/2.

2ème Méthode : Succession de soustraction (le nombre N à convertir est soustrait par le plus grand multiple 2).

Exemples :

- 25-16=9
  9-8=1
  1
  Donc 25_{10} = 1 1 0 0 1
- 37=32+5
  5=4+1
  1
  Donc 37_{10} = 1 0 0 1 0 1

Passage vers la base 8 :

$25_{10} = 31_8$
$423_{10} = 647_8$

Passage vers la base 16 :

$25_{10} = 19_{16} = 19_H$
$27_{10} = 1B_{16} = 1B_H$
$423_{10} = 1A7_{16} = 1A7_H$
On note que les restes supérieurs à 9 sont exprimés au moyen des lettres A à F.

3) Conversion octal-binaire et vice versa :

Chaque chiffre du nombre écrit dans le système octal est remplacé par son équivalent dans le système binaire à trois bits.

Exemple :

$2\ 5\ 7_8 = \underline{010}\ \underline{101}\ \underline{111}_2$

Pour convertir un nombre binaire en un nombre octal, il faut regrouper les 1 et 0 du nombre binaire par trois en allant vers la gauche à partir de la virgule, puis chaque groupe est remplacé par le chiffre octal correspondant.

Exemple :

$11001101111_2 = \underline{11}\ \underline{001}\ \underline{101}\ \underline{111} = 3157_8$

4) Conversion hexadécimal-binaire et vice versa :

Chaque chiffre du nombre écrit dans le système hexadécimal est remplacé par son équivalent dans le système binaire à quatre bits.

Exemple :

$9\ B\ 2_{16} = \underline{1001}\ \underline{1001}\ \underline{0010}_2$

Pour convertir un nombre binaire en hexadécimal, il faut regrouper les 1 et 0 du nombre binaire par quatre en allant vers la gauche à partir de la virgule, puis chaque groupe est remplacé par le chiffre hexadécimal correspondant.

Exemple :

$$001110100110_2 = \underline{0011}\ \underline{1010}\ \underline{0110} = 3A6_{16}$$

Remarque :

- Si i et g sont tous les deux des puissances de 2, on utilise la base 2 comme base de relais.
- Si i et g ne sont pas tous les deux des puissances de 2, on utilise la base 10 comme base de relais.

III. Arithmétique binaire :

On distingue quatre opérations arithmétiques binaires :

1) Addition binaire :

0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 0 et 1 de retenue (''carry'').

2) Multiplication binaire :

On multiplie les nombres binaires de la même façon qu'on multiplie les nombres décimaux.

$0 \times 0 = 0$
$0 \times 1 = 0$
$1 \times 0 = 0$
$1 \times 1 = 1$

La multiplication binaire est, en fait, une succession de décalage et des successions d'addition.

3) Division binaire :

Ceci revient à la Division Euclidienne :
0 : 0 = 0
0 : 1 = 0
1 : 0 = ∞
1 : 1 = 1

Applications :

- 1100 + 1010 = **10110**
- 1110111 + 100 + 1011 + 1001 = **10001111**
- 1100 $\times$ 111 = **1010100**
- 10101 $\times$ 101 = **1101001**

4) Soustraction :

$0-0=0$
$0-1=1$ et 1 d'empreinte (-2^1+1)
$1-0=1$
$1-1=0$

Remarque :

Lorsque le nombre A est inférieur au nombre B, la soustraction de $A-B$ se ramène à une addition$[A+(-B)]$.

IV. Représentation binaire des nombres signés :

Dans le système binaire, nous n'avons tenu compte jusqu'à maintenant que des nombres positifs. Un dispositif numérique doit traiter aussi bien les nombres positifs que négatifs, ce qui impose de prendre en compte deux symboles supplémentaires qui sont les signes + et -.

1) Représentation exacte :

Pour cela, un nombre binaire signé est composé de :

- Un ensemble de bits qui représente la valeur absolue du nombre.
- Un bit de signe indiquant si le nombre est positif ou négatif. Ce bit de signe est le premier à gauche de l'ensemble de bits représentant le nombre binaire. Lorsqu'il est égal à 0, le nombre est positif ; lorsqu'il est égal à 1, le nombre est négatif.

Les nombres écrits ainsi sont en notation exacte.

Exemples : Format 5 bits (1 bit pour le signe et 4 bits pour le nombre)

+ 8 **0** 1 0 0 0	+ 3 **0** 0 0 1 1
- 8 **1** 1 0 0 0	- 3 **1** 0 0 1 1

Remarque : La représentation se fait de $-(2^n-1) \rightarrow +(2^n-1)$

Inconvénient :

Cette représentation est simple, mais non convenable pour effectuer des calculs arithmétiques, car elle présente l'inconvénient d'avoir deux représentations possibles du zéro (+0 et -0). De plus, les calculateurs n'y ont

généralement pas recours, en raison de la complexité des circuits qui matérialisent cette notation.

2) Représentation en complément restreint (complément à 1) :

Un nombre binaire signé est écrit en notation en complément à 1, appelé aussi complément restreint, comme suit :

- Si le nombre est positif, il est écrit en nombre binaire pur avec un bit de signe 0, représenté par le bit de poids le plus fort.
- Si le nombre est négatif, il possède un bit de signe 1, mais écrit en notation en complément à 1.

Définissons alors la notation en complément à 1.
D'une manière générale, le complément à b d'un nombre N écrit dans la base b, noté N^*, est défini par la relation :

$$N^* = b^n - N$$

avec N^* : complément à b de N.
b : base du système de numération.
n : nombre de digit dans N.

Le complément restreint est noté CR(N), et consiste à complémenter le nombre bit à bit: $\overline{N}$. Par exemple, si le nombre $N = 11001$, alors $\overline{N} = 00110$.

Exemple : Format 4 bits

$X = 1001 \rightarrow CR(X) = 0110$

Utilité de cette représentation :

On a : **X + CR(X) = 2^n – 1**

$\rightarrow CR(X) + 1 = 2^n - X$ **Or** **2^n = 0**

Donc – X = CR(X) + 1

3) Représentation en complément vrai (complément à 2):

La représentation en complément à 2 (appelée aussi complément vrai et notée CV(N)) est très utile dans les opérations arithmétiques car avec, on peut faire une soustraction en effectuant en réalité une addition. Cela est très important dans le cas des ordinateurs et les calculateurs, puisqu'avec les mêmes circuits, on effectue l'addition et la soustraction.

Méthode pratique (appelée aussi indirecte) :

Une méthode pratique peut donner le complément à 2 d'un nombre binaire N en complémentant le nombre (complément à 1) et en ajoutant 1 au bit de poids le plus faible. On alors :

$$CV(N) = CR(N) + 1 = \overline{N} + 1 = -N$$

Méthode directe :

Cette méthode consiste à parcourir le nombre de droite à gauche, garder le 1er bit '1' rencontré et complémenter les bits suivants.

Exemple :

Format : 1 bit de signe + 6 bits pour N

$+23 \rightarrow 0010111$ avec 0 (1er bit à gauche) : bit de signe
$-23 \rightarrow 1101001$

Remarque :

- Pour revenir d'un nombre complémenté à 2 au nombre binaire initial, il faut de nouveau complémenter à 2.
- En effectuant l'opération A + CV(B) ; s'il y a un bit de dépassement, alors A>B ; sinon s'il n'y a pas de dépassement, alors A<B.
- En notation en complément à 2 et avec n bits, on représente les nombres signés compris dans l'intervalle $[-2^n, 2^n - 1]$.

A titre d'exemple pour n = 4 bits, on a le tableau 1.2 suivant :

Valeur décimale	Nombre binaire en complément à 2
$7 = 2^3 - 1$	0111
6	0110
5	0101
4	0100
3	0011
2	0010
1	0001
0	0000
-1	1111
-2	1110
-3	1101
-4	1100
-5	1011
-6	1010
-7	1001
$-8 = -2^3$	1000

Tableau 1.2 : Complément à 2 des nombres binaires à 4 bits.

4) Extension de signe :

Sur 4 bits :

$-3 =$ **1** 101 +3 = **0** 101

Sur 8 bits : → **on ajoute 4 fois le signe**

$-3=$ 1111 1101 +3 = 0000 0101

Sur 6 bits :

$-3=$ 11 1101 +3 = 00 0101

V. Notion de codage : Passage d'un code vers un autre

L'action de faire correspondre à des nombres, des lettres ou des mots, un groupe spécial de symboles s'appelle codage. Un code est un langage composé de différents symboles ou mots. Un mot est un ensemble de caractères numériques ou alphanumériques. On distingue deux types de codes :

- **Codes binaires pondérés :**

- Code binaire pur :

$2^n.....2^2 2^1 2^0$ → pondération

$\Rightarrow N = a_n.b^n + ... + a_0.b^0$

- Code hexadécimal :

$AF3_H = 10.16^2 + 15.16^1 + 3.16^0$ → pondération

- Code octal :

- Code BCD : décimal codé binaire (DCB)
  Binary Coded Decimal 8 4 2 1 : pondération

$\Rightarrow$ C'est un code pondéré qui représente chaque chiffre décimal par son équivalent binaire sur 4 bits, comme le montre le tableau 1.3 suivant :

Code décimal	Code B.C.D
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Tableau 1.3 : Code B.C.D

On note que les codes 1010, 1011, 1100, 1101, 1110, 1111 sont des combinaisons interdites, on les appelle des pseudo-tétrades.

Exemple :

Code décimal : 8 7 4

Code B.C.D : 1000 0111 0100 $\Rightarrow$ Juxtaposition des chiffres

On note bien la facilité relative avec laquelle on passe de ce code au nombre décimal correspondant et vice versa.

- Ajustement décimal / **Code BCD et addition binaire**

Règles d'ajustement :

- ❖ Ajouter 6 (0110) à chaque symbole dont l'équivalent binaire n'est pas un code BCD.
- ❖ En cas de retenue entre symboles, ajouter 6 (0110) au symbole qui a provoqué le débordement.

- **Codes binaires non pondérés :**

Dans lesquels on ne peut repérer le poids des chiffres binaires des groupes correspondants, et sont élaborés sur une base à développement mathématique complexe ou sont caractérisés par des tables faites spécialement.

- Code de Gray (code binaire réfléchi ou encore code miroir)

Le code Gray est un code qui appartient à la catégorie des codes dit à distance minimale, du fait qu'une représentation codée en Gray ne diffère de celle qui la précède que d'un élément binaire, comme le montre ce tableau.

Code décimal	Code binaire	Code Gray
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Tableau 1.4 : Code décimal, binaire et Gray

Pour n bits $\rightarrow$ on peut coder de 0 à $2^n - 1$ entiers.
D'après le code de Gray, on peut déduire le principe de la table de Karnough qu'on verra dans le Chapitre 2.

Le code de Gray peut être généré par réflexion (**effet miroir**). Les deux premiers nombres du code Gray sont 0000 et 0001. On obtient les deux nombres suivants en réfléchissant la combinaison par rapport au 1er axe de symétrie et en ajoutant un 1 au digit de rang immédiatement supérieur. Le même processus de réflexion aura lieu entre les chiffres trois et quatre (2ème axe de symétrie), puis entre sept et huit (3ème axe de symétrie), etc…

Conversion Gray/Décimal :

La règle est la suivante : $\pm(2^n - 1)$

Exemples :

100_{Gray} $\rightarrow$ **+ 2^3 – 1 = 7_{dec}**
111_{Gray} $\rightarrow$ **+ (2^3 – 1) – (2^2 – 1) + (2^1 – 1) = 7 – 3 + 1 = 5_{dec}**

Conversion Décimal/Gray :

- **1ère méthode**

$$Décimal \rightarrow Binaire \rightarrow \frac{N \oplus 2.N}{2}$$

- **2ème méthode**

On maintient le bit de poids le plus fort et on fait l'addition des autres bits un à un.

CHAP 2 : Eléments de L'Algèbre de Boole et Opérateurs logiques élémentaires

L'Algèbre de Boole est basé sur 3 opérateurs fondamentaux appelés ('ou', 'et', 'Non') qui sont de même nature que celles que l'on retrouve dans la théorie générale des ensembles.

I. Eléments de l'Algèbre de Boole :

Soit E = {0, 1}
Sur E, sont définies deux lois :

Somme logique (+)	Produit logique (.)
0 + 0 = 0	0 . 0 = 0
0 + 1 = 1	0 . 1 = 0
1 + 0 = 1	1 . 0 = 0
1 + 1 = 1	1 . 1 = 1
$\rightarrow$ Opérateur OU Logique	$\rightarrow$ Opérateur ET Logique
Notation : ''OU''	Notation : ''ET''

$*$) L'opérateur de complémentation (NON) :

$$\overline{0} = 1$$
$$\overline{1} = 0$$

II. Propriétés des lois des opérateurs logiques :

Pour effectuer tout calcul booléen, on utilise un ensemble de théorèmes.

Associativité	$(A.B).C = A.(B.C) = A.B.C$	$(A+B)+C = A+(B+C) = A+B+C$
Commutativité	$A.B = B.A$	$A+B = B+A$
Idempotence	A . A = A	$A+A = A$
Élément neutre	A . 1 = A	$A+0 = A$
Élément absorpant	A . 0 = 0	$A+1 = 1$

Distributivité :

$*$**)** A . (B + C) = (A . B) + (A . C)
$\Rightarrow$ Le produit logique est distributif par rapport à la somme logique.

$*$**)** A + (B . C) = (A + B) . (A + B) . (A + C)
$\Rightarrow$ La somme logique est distributive par rapport au produit logique.

Opérateur Complémentation :

$\overline{\overline{A}} = A$
$\overline{A} + A = 1$
$\overline{A}.A = 0$

III. Règles de calcul :

1) Règle d'allégement :

∗) $A.(\overline{A} + B) = A.B$
∗) $A + \overline{A}.B = A + B$

2) Règle d'absorption :

∗) $A.(A + B) = A$
∗) $A + A.B = A$

3) Théorèmes de De Morgan :

∗) $\overline{A+B} = \overline{A}.\overline{B} \rightarrow$ Le complément d'une somme logique est égale au produit des compléments.
∗) $\overline{A.B} = \overline{A} + \overline{B} \rightarrow$ Le complément d'un produit logique est égal à la somme des compléments.

IV. Opérateurs logiques élémentaires ;

1) Porte ET (AND) :

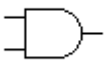
La porte ET est un circuit électronique qui réalise la fonction multiplication logique.

X = A.B

La table de vérité d'une porte ET à deux variables est donnée par le tableau ci-dessous.

A	B	X=A.B
0	0	0
0	1	0
1	0	0
1	1	1

Symbole :



2) Porte OU (OR) :

La porte OU est un circuit électronique qui réalise la fonction addition logique (+).

$$X = A + B$$

La table de vérité d'une porte OU à deux variables est donnée par le tableau ci-dessous.

A	B	X = A + B
0	0	0
0	1	1
1	0	1
1	1	1

$\underline{\underline{F = 1}}$ A = 0 et B = 1 ; A = 1 et B = 0 ; A = 1 et B = 1

$\Rightarrow F = \overline{A}.B + A.\overline{B} + A.B = B.(A + \overline{A}) + A.\overline{B} = B + A.\overline{B} = A + B$

On note qu'il suffit qu'un signal d'entrée soit à 1 pour que le signal de sortie le soit.

Symbole :



3) Porte NON (NOT) :

C'est une porte à une seule entrée, la sortie X est le complément de l'entrée A. Cette porte matérialise l'opérateur inverseur.

$$\underline{X = \overline{A}}$$

La table de vérité est donnée par le tableau ci-dessous.

A	X = $\overline{A}$
0	1
1	0

Symbole :



4) Les fonctions dérivées :

a) Porte NON-ET (NAND) :

La porte NON-ET est équivalente à une porte ET suivie d'un inverseur. L'expression de la fonction de sortie d'une porte NON-ET est :

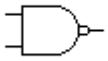
$$X = \overline{A.B}$$

La table de vérité de la porte NON-ET est donnée par le tableau ci-dessous.

A	B	$X = \overline{A.B}$
0	0	1
0	1	1
1	0	1
1	1	0

On note que le signal de sortie vaut 0 seulement si les deux signaux d'entrée sont simultanément à 1. Ce résultat peut s'étendre à une porte NON-ET à N entrées.

Symbole :



b) Porte NON-OU (NOR) :

La porte NON-OU appelée aussi porte NI est équivalente à une porte OU suivie d'un inverseur. L'expression de sortie d'une fonction NON-OU est :

$$X = \overline{A + B}$$

La table de vérité de la porte NON-OU est donnée par le tableau ci-dessous.

A	B	$X = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

Symbole :



c) Porte OU-exclusif (XOR) :

Une porte OU-exclusif à deux entrées A et B est définie algébriquement par la relation suivante :

$$X = A \oplus B = \overline{A}.B + A.\overline{B}$$

Le signe $\oplus$ représente l'opération OU-exclusif.
$\oplus$: Double somme $\Rightarrow$ fonction symétrie.

La table de vérité d'une porte OU-exclusif est donnée comme suit :

A	B	$X = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

La porte OU-exclusif met la sortie à 1 quand les signaux sur ses deux entrées sont différents.

Symbole :



Propriétés de l'opérateur XOR :

La porte OU-exclusif vérifie les propriétés suivantes :

- $A \oplus B = B \oplus A$ (commutativité)
- $(A \oplus B) \oplus C = A \oplus (B \oplus C)$ (associativité)
- $A \oplus A = 0$
- $A \oplus \overline{A} = 1$
- $A \oplus 0 = A$
- $A \oplus 1 = \overline{A}$

D'après les deux dernières propriétés, on peut dire que la porte XOR peut servir comme un inverseur commandé à 1 bit.

Applications :

La porte OU-exclusif permet :

- La détection de deux éléments binaires différents :
  $A \oplus B = 1$ si $A \neq B$
  $A \oplus B = 0$ si $A = B$
- La détection d'un nombre de variables impair (X=1 si A=1 ou B=1 mais pas les deux.
- L'addition de deux éléments binaires (additionneur binaire) :
  $0 \oplus 0 = 0$
  $0 \oplus 1 = 1$
  $1 \oplus 0 = 1$
  $1 \oplus 1 = 0$ il existe en plus dans cette combinaison une retenue.